How to Approach Projects Using AI Agents?

Part 1: How to Talk to the AI (Building the App)

When you ask an AI to build something, don't just say, "Build me an app." You have to give it a clear **Context, Requirement, and Constraint (CRC)**.

Let's imagine you are buil,ding a simple **College Book Buy/Sell App** for your campus.

Step A: Give the AI the Full Picture (CRC Prompt)

- **Context:** "I am building a web app for college students to buy and sell used textbooks using React and Node.js."
- **Requirement:** "Write the database structure for the 'Users' (students) and 'Books' (inventory). Make sure they are linked together."
- **Constraint:** "Make sure we don't repeat data. For example, don't save a student's phone number on every single book they upload. Save their phone number once in a 'Students' list, and just link their Student ID to the books they are selling."

> **Why this matters:** If you don't explicitly tell the AI to connect the features in the backend database, it will just draw pretty buttons on the screen that don't actually do anything when you click them.

Step B: Build it Like Lego Blocks Do not ask the AI to build the whole app at once. Ask for one piece at a time.

- **Prompt 1:** "First, build the login system. Create two types of users: 'Admins' (App Owners) and 'Students' (Regular Users)."
- **Prompt 2:** "Great. Now, using the rules from Step 1, create a feature where *only* Admins can delete a book from the system."

Part 2: How to Understand the Code

When looking at the code the AI generates, don't try to read every single word. Look for the core logic and the "Key Lines."

1. Follow the Data

Instead of memorizing code, track where a piece of data goes. Imagine a student typing in the price of a book they want to sell:

- **In-flow:** They type "$50" into a form on the website.
- **Transformation:** The code might add a $2 app fee, making it "$52".
- **Storage:** The code saves "$52" into your database.
- **Out-flow:** The website displays "$52" to other students looking at the book.

The Blue Key Line (Storage): Look for words like .save(), .create(), or INSERT INTO. That is the exact moment the data is permanently saved to your database.

2. Safety Checks

You have to find where your app protects itself from bad data or bad users. These are the "guards" in your code.

- **Validation (Stopping Bad Data):** What stops a student from pricing a book at $-10 or typing a 5,000-word essay into the "Title" box?
- **Authorization (Stopping Bad Users):** What stops a regular student from pressing the "Delete All Users" button?

The Green Key Line (Guards): Look for if statements at the very top of a function (e.g., if (price < 0) { stop } or if (user != 'admin') { stop }).

3. Keep Things Separate (Modularity)

Why isn't all the code in one giant file?

Imagine a car: the engine and the radio are separate. If the radio breaks, the car still drives. In your app, the "Login Code" and the "Search for Books Code" should be in different files. If the search bar breaks, students should still be able to log in.

The Purple Key Line (Connections): Look for import or require at the very top of your files. This shows you how the different "Lego blocks" of your app are connected to each other.

4. Planning for the Worst (Error Handling)

Your code won't always work perfectly. What happens if the college Wi-Fi goes down exactly when a student tries to upload a book?

Good code doesn't just crash and show a scary blank screen. It catches the problem and shows a nice message like, "Oops, connection lost! Please try again."

The Red Key Line (The Safety Net): Look for the catch or except blocks. This line means the AI specifically planned for a worst-case scenario.